有限元编程

cdk

2024 年 4 月 24 日

目录

1 2020.11.29 有限元编程前期准备 **2**
1.1 来源和思路 . . . 2
1.2 目标 . . . 2
1.3 安排与方法 . . . 2
1.4 概念化的指导性编程 . . . 2

2 2020.12.06 第一章关于一维二阶椭圆方程的有限元 **3**
2.1 弱格式/Galerkin 格式 . . . 3
2.1.1 目标问题 . . . 3
2.1.2 弱形式 . . . 3
2.1.3 一维 Sobolev spaces . . . 4
2.1.4 Galerkin 格式 . . . 5
2.2 有限元空间 . . . 5
2.2.1 网格 Mesh . . . 5
2.2.2 一维有限元空间 . . . 6
2.3 有限元离散 . . . 6
2.3.1 离散格式 . . . 6
2.3.2 刚度矩阵的装配 . . . 7
2.3.3 载荷向量装配 . . . 9
2.4 边界处理 . . . 9
2.5 有限元方法综合 . . . 10
2.5.1 基础算法 . . . 10
2.5.2 有限元通用框架结构 . . . 10
2.5.3 网格信息矩阵 . . . 11
2.5.4 有限元信息矩阵 . . . 11
2.5.5 局部装配 . . . 12
2.5.6 施加边界条件 . . . 15
2.6 推广 . . . 15
2.7 总结 . . . 15

A 2020.12.13 第一次编程课 **16**

1 2020.11.29 有限元编程前期准备

1.1 来源和思路

- 要准备自己的程序包, 这样才能更有效的更改代码完成不同的研究课题.
- 首先准备 possion 方程的程序包, 在推广到线弹性方程程序包.
- 主要是一种概念程序包, 有利于修改但是不适合运用于大规模运算.
- 对算法和程序通用性的理解.
- 有限元数据结构和思想都是类似或者一样的.
- 两个层次: Lin Tao 计算数学研究生入门有限元程序 (更加基础).

1.2 目标

- 有限元算法构造吃透, 特别是通用性.
- 自己从零开始构建程序包, 不借助外力不和人讨论不过早地看别人的程序.

1.3 安排与方法

- 逐渐积累, 在课上翻译 50% 的程序, 剩下的基本补完. 石墨文档收集了一些问题.

1.4 概念化的指导性编程

- 将公式和程序对应起来, 牺牲大规模运算以及物理意义等, 使得修改程序时更加方便.

fig:sec1_1

2 2020.12.06 第一章关于一维二阶椭圆方程的有限元

前提

- 使用一维程序只是为了讲解方便, 并不一定要使用通用性结构, 一般也都能实现. 主要是为了理解通用性程序.
- 主要讲前五个小节.

2.1 弱格式/Galerkin 格式

2.1.1 目标问题

求解目标问题:

$$-\frac{d}{dx}\left(c(x)\frac{du(x)}{dx}\right)=f(x), a<x<b,$$
$$u(a)=g_a, u(b)=g_b$$

一维稳态线性二阶椭圆狄利克雷边界条件线性元求解. [1]
先讲不动用复杂结构的程序, 再由此推广为何需要引入复杂数据结构.

2.1.2 弱形式

使用变分法构造弱格式.[2]
在原式两边同乘函数 $v(x)$ 得到

$$-\frac{d}{dx}\left(c(x)\frac{du(x)}{dx}\right)v(x)=f(x)v(x), a<x<b.$$

两边积分得

$$-\int_a^b\frac{d}{dx}\left(c(x)\frac{du(x)}{dx}\right)v(x)dx=\int_a^b f(x)v(x)dx.$$

[1]有限元最简单的情形

[2]$u(x)$ 称为 trial 函数, $v(x)$ 称为 test 函数. 且 $v(x)$ 是自行选取的 (零时无意义), 如何选取 test 函数?

两边分部积分

$$
\begin{aligned}
&\int_a^b \frac{d}{dx}\left(c(x)\frac{du(x)}{dx}\right)v(x)dx \\
=&\int_a^b (cu')' vdx \\
=&\int_a^b vd(cu') \\
=&cu'v|_a^b - \int_a^b cu'dv \\
=&c(b)u'(b)v(b) - c(a)u'(a)v(a) - \int_a^b cu'v'dx.
\end{aligned}
$$

则有

$$
-c(b)u'(b)v(b) + c(a)u'(a)v(a) + \int_a^b cu'v'dx = \int_a^b fvdx.
$$

因此会带来一些边界项也就需要处理边界条件. 这里我们先处理狄利克雷[3]条件边界[4]. Dirichlet 边界条件不是自然边界条件, 被称为本质[5]边界条件.

因为 Dirichlet 边界条件已知边界上的值, 所以是不需要求解的. 因此可以选取合适的 test 函数 $v(x)$ 来施加边界条件[6].

在本问题中即选取 test 函数 $v(x)$ 使得 $v(a) = v(b) = 0$.[7]

因此变分形式化简为:

$$
\int_a^b cu'v'dx = \int_a^b fvdx.
$$

但是需要考虑的是 u, v 应该属于何种空间[8]? 即 Sobolev spaces![9]

2.1.3 一维 Sobolev spaces

dfn:support **Definition 1** (函数的支撑集 support)**.** *If u is a function, then its support supp(u) is the closure of the set on which u is nonzero.*

注记 1. 所谓支撑集就是函数值不为零的区域的闭包.

dfn:comp_supp **Definition 2** (紧支集 Compactly supported)**.** *If u is a function defined on an open interval I and supp(u) is a compact subset (that is, a closed and bounded subset), then u is said to be compactly supported in I.*

注记 2. 在欧式空间中紧集等价于有界闭. 但实际紧集的定义是有有限开覆盖.

lem:chap1_1 **Lemma 1** (紧支集函数边界为零)**.** *A function compactly supported in an open interval I is zero on and near the boundary of I.*

dfn:C0infty **Definition 3** (紧支光滑函数空间 C_0^∞)**.** *$C_0^\infty(I)$ is the set of all functions that are infinitely differentiable on I and compactly supported in I.*

[3] Dirichlet

[4] 在扩展中会讲解对于 Neumann 边界条件和 Robin 边界条件的处理方法. 也就是对自然边界条件的处理

[5] essential

[6] 或者说是将边界消去.

[7] 这需要在程序中体现出来.

[8] 或者说 u, v 需要有哪些性质.

[9] 最好掌握, 主要涉及理论, 与程序实现关系不大. 因为在离散情形下, 每个单元上都是分片多项式, 多项式的弱导数就是强导数没有区别!

则对于 $v \in C_0^\infty(I)$, 有 $v(a) = v(b) = 0$. 从而

$$\int_a^b u'v dx = -\int_a^b uv' dx.$$

从而引入弱导数的概念. 这是因为当导数不存在时上述等式不可能成立, 但是我们仍有希望找到满足上式的 w, 虽然其不是 u 的强导数.

dfn:weak_d **Definition 4** (弱导数)**.** *Suppose u is a real-valued function defined on an open interval $I = (a, b)$ and that u is integrable over every compact subset of I. If there exists another locally integrable function w defined on I such that*

$$\int_a^b wv dx = -\int_a^b uv' dx.$$

for all $v \in C_0^\infty(I)$, then u is said to be weakly differentiable and $w$is called the weak derivative of u.

em:weak_donly **Lemma 2** (弱导数唯一)**.** *If u is differentiable, then u is weakly differentiable and its weak derivative is u'.*

wait:1 **待续...** 定义一维空间上的 L^2, H^1, H_0^1 函数空间.

注记 3. 无聊的内容重述, 具体可见讲义 chapter1.page13

2.1.4 Galerkin 格式

首先只考虑协调元的情形.
假设有一个有限维的子空间 $U_h \subset H^1(a, b)$. 类似的有定义 U_{h0}[10].
Galerkin 格式: find $u_h \in U_h$ such that

$$\begin{aligned} a(u_h, v_h) &= (f, v_h) \\ \Leftrightarrow \int_a^b cu_h' v_h' dx &= \int_a^b f v_h dx \end{aligned}$$

for any $v_h \in U_{h0}$.
基本思想: 使用有限维空间去逼近无穷维空间.
问题在于如何找到有限维空间 U_h.[11]

2.2 有限元空间

2.2.1 网格 Mesh

Assume that we have a uniform partition of $[a, b]$ into N elements with mesh size $h = \frac{b-a}{N}$.
Let $x_i = a + (i-1)h (i = 1, \cdots, N+1)$ denote the mesh nodes.
Let $E_n = [x_n, x_{n+1}] (n = 1, \cdots, N)$ denote the mesh elements.

注记 4. 一维区间上步长、节点以及单元的定义. 这里网格使用的是均匀网格, 实际上可能会采用非均匀网格, 比如在使用自适应算法时, 单元长度就可能会不一样.

[10] 即 U_h 在 Dirichlet 边界为零的函数全体构成的子空间.
[11] 具体构造在扩展中会提及.

2.2.2 一维有限元空间

从而一维线性有限元空间 U_h 就是基于网格[12]的分片线性[13]多项式构成的空间.

$$U_h = \{\phi \in C[a,b] : \phi(x) \text{ is linear on each } [x_n, x_{n+1}], (n = 1, 2, \cdots, N)\}.$$

thm:dimdfe **Theorem 1** (一维离散有限元空间维数)**.** *U_h 是 $C[a,b]$ 的 $N+1$ 维子空间.*

注记 5. 证明分为两部分. 易证其实子空间. 其次只要找到 $N+1$ 个基函数刚好构成 U_h 的基底即可.
考虑 $\phi_j(x) \in U_h$ 使得

$$\phi_j(x_i) = \delta_{ij} = \begin{cases} 0, & \text{if } j \neq i, \\ 1, & \text{if } j = i. \end{cases}$$

对于 $i, j = 1, \cdots, N+1$ 成立.
事实上就是

$$\phi_1(x) = \begin{cases} \frac{x_2 - x}{h}, & \text{if } x_1 \leq x \leq x_2, \\ 0, \text{ otherwise}, \end{cases}$$

$$\phi_j(x) = \begin{cases} \frac{x - x_{j-1}}{h}, & \text{if } x_{j-1} \leq x \leq x_j, \\ \frac{x_{j+1} - x}{h}, & \text{if } x_j \leq x \leq x_{j+1}, \\ 0, \text{ otherwise}, \end{cases}$$

$$(j = 2, \cdots, N)$$

$$\phi_{N+1}(x) = \begin{cases} \frac{x - x_N}{h}, \text{ if } x_N \leq x \leq x_{N+1}, \\ 0, \text{ otherwise}, \end{cases}$$

基底证明细节可看讲义 chapter1.page18-21.

2.3 有限元离散

2.3.1 离散格式

回顾 Galerkin 格式 (暂时不考虑 Dirichlet 边界条件) find $u_h \in U_h$ such that

$$a(u_h, v_h) = (f, v_h)$$
$$\Leftrightarrow \int_a^b c u_h' v_h' dx = \int_a^b f v_h dx$$

for any $v_h \in U_h$.
因为 $u_h \in U_h = \text{span}\{\phi_j\}_{j=1}^{N+1}$, 则

$$u_h = \sum_{j=1}^{N+1} u_j \phi_j$$

对于某些系数 $u_j (j = 1, \cdots, N+1)$.
需要求解这 $N+1$ 个参数, 这需要构造一个线性方程组来求解.
因此我们可以选取 test 函数 $v_h = \phi_i \, (i = 1, \cdots, N+1)$.

[12]是指在单元片上线性.

[13]线性主要是指多项式次数小于等于一. 实际上高次元也有一般计算量更大但是精度更好. 这里选择的是最简单的线性元以方便讲解.

则有限元离散格式就有

$$\int_a^b c\left(\sum_{j=1}^{N+1} u_j\phi_j\right)' \phi_i' dx = \int_a^b f\phi_i dx, i=1,\cdots,N+1$$

$$\Rightarrow \sum_{j=1}^{N+1} u_j \left[\int_a^b c\phi_j'\phi_i' dx\right] = \int_a^b f\phi_i dx, i=1,\cdots,N+1.$$

定义刚度矩阵[14]

$$A=[a_{ij}]_{i,j=1}^{N+1} = \left[\int_a^b c\phi_j'\phi_i' dx\right]_{i,j=1}^{N+1}.$$

定义载荷向量[15]

$$\vec{b}=[b_i]_{i=1}^{N+1} = \left[\int_a^b f\phi_i dx\right]_{i=1}^{N+1}.$$

定义未知向量

$$\vec{X}=[u_j]_{j=1}^{N+1}.$$

从而有线性代数方程组

$$A\vec{X}=\vec{b}.$$

Remark 1. *如果初始椭圆方程是对称正定的那么 A 也是对称正定的.*

Remark 2. *In fact, since*

$$\phi_j(x_k)=\delta_{jk}=\begin{cases}0, & \text{if } j\neq k\\ 1, & \text{if } j=k.\end{cases}$$

then

$$u_h(x_k)=\sum_{j=1}^{N+1} u_j\phi_j(x_k)=u_k.$$

Hence the coefficient u_j is actually the numerical solution at the node $x_j (j=1,\cdots,N+1)$.
Once $\vec{X}=[u_j]_{j=1}^{N+1}$ is obtained, the finite element solution $u_h=\sum_{j=1}^{N+1} u_j\phi_j$ and the numerical solutions at all the mesh nodes are obtained.

注记 6. 由于使用的是线性单元, 未知量的值刚好是数值解在节点处的值.

2.3.2 刚度矩阵的装配

在本节中将介绍一种特殊方法来装配矩阵和向量, 在后续章节subsec:FEmethod2.5中将讨论不同的更一般的框架.
由 $\phi_j\ (j=1,\cdots,N+1)$ 的定义可知, ϕ_j 仅在与节点 x_j 相邻的单元上非零且在其他单元上为零.[16]
因此考虑[17]

$$a_{ij}=\int_a^b c\phi_j'\phi_i' dx = \sum_{n=1}^{N}\int_{x_n}^{x_{n+1}} c\phi_j'\phi_i' dx, i,j=1,\cdots,N+1.$$

[14] stiffness matrix
[15] load vector
[16] 即值域只可能属于 $[x_{j-1},x_{j+1}]$, 边界处则分别属于 $[x_1,x_2]$ 和 $[x_j,x_{j+1}]$.
[17] 这里需要有一组遍历以及对应的局部积分. 但是事实上很多节点处的局部积分都是零.

注记 7. 核心思想单刚组装总刚.

下面对于各类情况分别进行讨论:

情形一:

当 $|i-j|>1$ 时, 即 x_i 和 x_j 不是相邻节点. 则在任意单元 $[x_n, x_{n+1}](n=1,2,\cdots,N)$ 上 ϕ_j 和 ϕ_i 中至少有一个是 0.

即有公式:

$$\begin{aligned}
&\int_{x_n}^{x_{n+1}} c\phi_j'\phi_i' dx = 0(n=1,\cdots,N) \\
\Rightarrow \quad & a_{ij} = \sum_{n=1}^{N} \int_{x_n}^{x_{n+1}} c\phi_j'\phi_i' dx = 0.
\end{aligned}$$

情形二:

当 $i=j+1\,(j=1,2,\cdots,N)$ 时, 唯一使得 ϕ_j 和 ϕ_i 都非零的单元是 $[x_j, x_{j+1}]$.

从而有

$$\begin{aligned}
&\int_{x_n}^{x_{n+1}} c\phi_j'\phi_i' dx = 0(n=1,\cdots,j-1,j+1,\cdots,N) \\
&\Rightarrow a_{ij} = \sum_{n=1}^{N} \int_{x_n}^{x_{n+1}} c\phi_j'\phi_i' dx = \int_{x_j}^{x_{j+1}} c\phi_j'\phi_i' dx \\
\Rightarrow \quad & a_{j+1,j} = \int_{x_j}^{x_{j+1}} c(x) \left(\frac{x_{j+1}-x}{h}\right)' \left(\frac{x-x_j}{h}\right)' dx \\
&= -\frac{1}{h^2} \int_{x_j}^{x_{j+1}} c(x) dx.
\end{aligned}$$

情形三:

当 $i=j-1\,(j=2,\cdots,N+1)$ 时, 唯一使得 ϕ_j 和 ϕ_i 都非零的单元是 $[x_{j-1}, x_j]$.

$$\begin{aligned}
&\int_{x_n}^{x_{n+1}} c\phi_j'\phi_i' dx = 0(n=1,\cdots,i-1,i+1,\cdots,N) \\
&\Rightarrow a_{ij} = \sum_{n=1}^{N} \int_{x_n}^{x_{n+1}} c\phi_j'\phi_i' dx = \int_{x_{j-1}}^{x_j} c\phi_j'\phi_i' dx \\
\Rightarrow \quad & a_{j-1,j} = \int_{x_{j-1}}^{x_j} c(x) \left(\frac{x-x_{j-1}}{h}\right)' \left(\frac{x_j-x}{h}\right)' dx \\
&= -\frac{1}{h^2} \int_{x_j-1}^{x_j} c(x) dx.
\end{aligned}$$

情形四:

当 $i=j\,(j=2,\cdots,N)$ 时, 使得 ϕ_j 和 ϕ_i 都非零的单元是 $[x_{j-1}, x_j]$ 以及 $[x_j, x_{j+1}]$.

$$\begin{aligned}
&\int_{x_n}^{x_{n+1}} c\phi_j'\phi_i' dx = 0(n=1,\cdots,j-2,j+1,\cdots,N) \\
&\Rightarrow a_{ij} = \sum_{n=1}^{N} \int_{x_n}^{x_{n+1}} c\phi_j'\phi_i'\ dx = \int_{x_{j-1}}^{x_j} c\phi_j'\phi_i'\ dx + \int_{x_j}^{x_{j+1}} c\phi_j'\phi_i'\ dx
\end{aligned}$$

$$
\begin{aligned}
\Rightarrow a_{jj} &= \int_{x_{j-1}}^{x_j} c(x)\left(\frac{x-x_{j-1}}{h}\right)'\left(\frac{x-x_{j-1}}{h}\right)' dx \\
&+ \int_{x_j}^{x_{j+1}} c(x)\left(\frac{x_{j+1}-x}{h}\right)'\left(\frac{x_{j+1}-x}{h}\right)' dx \\
&= \frac{1}{h^2}\int_{x_{j-1}}^{x_j} c(x)\ dx + \frac{1}{h^2}\int_{x_j}^{x_{j+1}} c(x)\ dx.
\end{aligned}
$$

情形五:

当 $i=j=1$ 时, 使得 ϕ_j 和 ϕ_i 都非零的单元是 $[x_1,x_2]$.

从而有

$$
\begin{aligned}
&\int_{x_n}^{x_{n+1}} c\phi_1'\phi_1'\ dx = 0\ (n=2,\cdots,N) \\
\Rightarrow a_{11} &= \sum_{n=1}^{N}\int_{x_n}^{x_{n+1}} c\phi_1'\phi_1' dx = \int_{x_1}^{x_2} c\phi_1'\phi_1' dx \\
\Rightarrow a_{11} &= \int_{x_1}^{x_2} c(x)\left(\frac{x_2-x}{h}\right)'\left(\frac{x_2-x}{h}\right)' dx \\
&= \frac{1}{h^2}\int_{x_1}^{x_2} c(x)\ \mathrm{d}x.
\end{aligned}
$$

情形六:

当 $i=j=N+1$ 时, 使得 ϕ_j 和 ϕ_i 都非零的单元是 $[x_N,x_{N+1}]$.

从而有

$$
\begin{aligned}
&\int_{x_n}^{x_{n+1}} c\phi_{N+1}'\phi_{N+1}'\ dx = 0\ (n=1,\cdots,N-1) \\
\Rightarrow a_{N+1,N+1} &= \sum_{n=1}^{N}\int_{x_n}^{x_{n+1}} c\phi_{N+1}'\phi_{N+1}' dx = \int_{x_N}^{x_{N+1}} c\phi_{N+1}'\phi_{N+1}' dx \\
\Rightarrow a_{N+1,N+1} &= \int_{x_N}^{x_{N+1}} c(x)\left(\frac{x-x_N}{h}\right)'\left(\frac{x-x_N}{h}\right)' dx \\
&= \frac{1}{h^2}\int_{x_N}^{x_{N+1}} c(x) dx.
\end{aligned}
$$

上述讨论了所有刚度矩阵元素, 且由此可知刚度矩阵稀疏对称且主要计算的是局部刚度矩阵.

最基本的算法是遍历单元计算矩阵的每个元素.

2.3.3 载荷向量装配

类似于刚度矩阵的讨论, 分为三种情况讨论即可, 分别为内部单元和在两端的边界单元.

详情可看讲义 chapter1.page36-39.

注记 8. 这是由于选取的形函数是紧致的, 所以只会在局部非零. 从而可以对全局积分做局部化.

2.4 边界处理

这里暂时只考虑 Dirichlet 边界条件.

首先由于 Dirichlet 边界条件, 我们已知在 $u(a)=g_a, u(b)=g_b$, 即已知解在 $x_1=a$ 和 $x_{N+1}=b$ 处的解.

主要处理的一种办法.

首先对于 $u_h=\sum_{j=1}^{N+1} u_j\phi_j$ 已知有限元解的系数 u_j 是在节点 $x_j(j=1,\cdots,N+1)$ 处的数值解. 现在我们已经确切地知道 $u_1=u(a)=g_a$ 和 $u_{N+1}=u(b)=g_b$. 可以直接将该值带入方程组简化方程. 但是程序会更加麻烦.

一个更加简便的方法是:

直接将方程组中第一和最后一个方程替换为如下公式:

$$u_1 = g_a \Rightarrow 1 \cdot u_1 + 0 \cdot u_2 + \cdots + 0 \cdot u_{N+1} = g_a,$$
$$u_{N+1} = g_b \Rightarrow 0 \cdot u_1 + \cdots + 0 \cdot u_N + 1 \cdot u_{N+1} = g_b.$$

对应到程序实现: 就是将刚度矩阵中的第一行和最后一行换成 $(1, 0, \cdots, 0)$ 和 $(0, \cdots, 0, 1)$. 同时将载荷向量的第一和最后一个变量修改为 g_a 和 g_b.
即伪代码为:

$$\begin{aligned}
&A(1,:) = 0;\\
&A(1,1) = 1;\\
&A(N+1,:) = 0;\\
&A(N+1,N+1) = 1;\\
&b(1) = g_a;\\
&b(N+1) = g_b;
\end{aligned}$$

注记 9. 这其实是在进行一些向量化编程. 其实 Dirichlet 边界条件是最后处理的, 比较麻烦的其实是剩下两种.

2.5 有限元方法综合

sec:FEmethod

2.5.1 基础算法总结

综上所述这样一个简单的有限元程序主要是以以下步骤来进行的:

- 输入 a, b, N. 计算步长 $h = \frac{b-a}{N}$ 和 $x_j = a + (j-1)h\,(j = \dot{1}, \cdots, N+1)$.
- 组装刚度矩阵 A.
- 组装载荷向量 $\vec{b}$.
- 将 Dirichlet 边界条件施加到方程中.
- 使用直接求解法或者迭代法求解方程组 $A\vec{X} = \vec{b}$.

注记 10. 注意到利用上述方法编写程序并不适合处理各种类型的偏微分方程. 我们需要寻求一种更一般的框架.

2.5.2 有限元通用框架结构

- 生成信息矩阵[18]: P, T, E[19].
- 装配矩阵和向量仅基于信息矩阵 P, T, E.[20]
- 类似的处理边界条件.
- 求解线性方程组: 数值线性代数.

[18] information matrice

[19] 如果不涉及单元边线积分, 则 E 基本不会用到.

[20] 事实上这就说明我们需要设计更好的数据结构以及相应更好的遍历方式. 基本思路是希望能够实现全自动地组装矩阵而不是和前面章节一样 case by case 地去分类讨论.

2.5.3 网格信息矩阵

首先定义相应的记号:

- 记 N 为网格单元的个数, 记 N_m 为网格节点的个数, 则有 $N_m = N+1$.
- 定义矩阵 P[21] 是所有网格节点的坐标矩阵.[22]
- 定义矩阵 T[23] 是所有网格单元上的网格节点的全局编号矩阵.[24]

对于一维情形相应的 P, T 矩阵为:

$$
\begin{aligned}
P &= \begin{pmatrix} x_1 & x_2 & \cdots & x_{N_m-1} & x_{N_m} \end{pmatrix} \\
&= \begin{pmatrix} x_1 & x_2 & \cdots & x_N & x_{N+1} \end{pmatrix}, \\
T &= \begin{pmatrix} 1 & 2 & \cdots & N_m-2 & N_m-1 \\ 2 & 3 & \cdots & N_m-1 & N_m \end{pmatrix} \\
&= \begin{pmatrix} 1 & 2 & \cdots & N-1 & N \\ 2 & 3 & \cdots & N & N+1 \end{pmatrix}.
\end{aligned}
$$

注记 11. 矩阵 T 中存储的内容是全局编号, 存储的规则是局部编号. 一维一致网格是最简单的情形.

2.5.4 有限元信息矩阵

上述网格信息矩阵 P, T 是基于网格节点的. 但是需要区分网格概念和有限元概念. 虽然在线性元的情况下网格节点就是有限元节点[25], 但是其他情形例如二次元就不是这样, 网格节点与有限元节点并不一致. 节点不一致相应的编号坐标就都不一致.
因此需要定义基于有限元节点的有限元信息矩阵 P_b 和 T_b, 其中下标 b 代表有限元基函数 (basis function).
记 N_b 为所有有限元基函数的个数 (= 未知变量个数 = 有限元节点的个数)[26]. 此时有 $N_b = N+1$.
则有

$$
u_h = \sum_{j=1}^{N_b} u_j \phi_j.
$$

对于线性有限元, 我们有 $P_b = P, T_b = T$. 即

$$
\begin{aligned}
P_b &= \begin{pmatrix} x_1 & x_2 & \cdots & x_{N_b-1} & x_{N_b} \end{pmatrix} \\
&= \begin{pmatrix} x_1 & x_2 & \cdots & x_N & x_{N+1} \end{pmatrix}, \\
T_b &= \begin{pmatrix} 1 & 2 & \cdots & N_b-2 & N_b-1 \\ 2 & 3 & \cdots & N_b-1 & N_b \end{pmatrix} \\
&= \begin{pmatrix} 1 & 2 & \cdots & N-1 & N \\ 2 & 3 & \cdots & N & N+1 \end{pmatrix}.
\end{aligned}
$$

在第二章中将会介绍更复杂的二次元 (quadratic element).

[21] 节点坐标阵

[22] 即 P 的第 k 列存储的是第 k 个网格节点的坐标.

[23] 单元节点编号阵

[24] 即 T 矩阵的第 k 列存的是第 k 个单元所有顶点的全局编号.

[25] 定义有限元基函数时用到的节点. 对于线性元不用区分是因为所有线性基函数都是基于网格节点构造的. 且有限元节点与线性基函数一一对应.

[26] 基于线性元的前提下.

2.5.5 局部装配

在装配刚度矩阵时:

- 刚度矩阵 A 中的所有非零元素都来自与定义在网格单元上的非零局部积分.
- 在每个非零局部积分中, trial 和 test 函数都与积分区间 (其实就是某个单元) 的节点相对应的[27].
- 在任意单元上, 所有非零局部积分都对刚度矩阵相应的非零项做出非平凡的贡献.

新的装配刚度矩阵 A 的想法是:

- 遍历所有的网格单元.
- 在每个单元上计算所有的非零局部积分.
- 将这些非零局部积分装配经刚度矩阵 A 的相应项中.

注记 12. 这就是单刚组装总刚的具体实现.

在每个单元上计算所有的非零局部积分:
在第 n 个单元 $E_n = [x_n, x_{n+1}]$ 上, 仅在 trial 和 test 函数都对应该单元的有限元节点时可以得到非零局部积分.
即我们仅考虑 trial 和 test 函数是 ϕ_n 和 ϕ_{n+1} 的情形.
那么在单元 E_n 上关于全局基函数 ϕ_n 和 ϕ_{n+1} 就仅有四个非零局部积分:

$$\int_{x_n}^{x_{n+1}} c\phi_n'\phi_n' dx, \int_{x_n}^{x_{n+1}} c\phi_{n+1}'\phi_n' dx, \int_{x_n}^{x_{n+1}} c\phi_n'\phi_{n+1}' dx, \int_{x_n}^{x_{n+1}} c\phi_{n+1}'\phi_{n+1}' dx.$$

简记为:

$$\int_{x_n}^{x_{n+1}} c\phi_j'\phi_i' \text{ dx } (i, j = n, n+1).$$

回顾基函数:

$$\phi_1(x) = \begin{cases} \frac{x_2 - x}{h}, & \text{if } x_1 \le x \le x_2, \\ 0, \text{ otherwise}, \end{cases}$$

$$\phi_j(x) = \begin{cases} \frac{x - x_{j-1}}{h}, & \text{if } x_{j-1} \le x \le x_j, \\ \frac{x_{j+1} - x}{h}, & \text{if } x_j \le x \le x_{j+1}, \\ 0, \text{ otherwise}, \end{cases}$$

$$(j = 2, \cdots, N)$$

$$\phi_{N+1}(x) = \begin{cases} \frac{x - x_N}{h}, \text{ if } x_N \le x \le x_{N+1}, \\ 0, \text{ otherwise}, \end{cases}$$

定义局部线性基函数:

$$\psi_{n1} = \phi_n|_{E_n} = \frac{x_{n+1} - x}{h}, \quad \psi_{n2} = \phi_{n+1}|_{E_n} = \frac{x - x_n}{h}.$$

在一个单元上, 局部基函数的个数 $N_{lb} = 2$.
则唯一四个非零局部积分改写成:

$$\int_{x_n}^{x_{n+1}} c\psi_{n1}'\psi_{n1}' dx, \int_{x_n}^{x_{n+1}} c\psi_{n2}'\psi_{n1}' dx, \int_{x_n}^{x_{n+1}} c\psi_{n1}'\psi_{n2}' dx, \int_{x_n}^{x_{n+1}} c\psi_{n2}'\psi_{n2}' dx.$$

[27] 对应是指在该节点处为 1 在其它节点处为 0.

用局部基函数 ψ_{n1} 和 ψ_{n2} 简记为:

$$\int_{x_n}^{x_{n+1}} c\psi'_{n\alpha}\psi'_{n\beta}dx\,(\alpha,\beta=1,2).$$

如何计算积分?

Gauss qaudrature 高斯积分公式

装配总刚度矩阵主要是将单元刚度矩阵对应到相应节点处即可.

使用有限元信息矩阵 T_b 可以找到局部 trial 和 test 函数的全局节点编号.[28]

T_b 矩阵存储有限元单元上节点的全局编号:

$$T_b = \begin{pmatrix} 1 & 2 & \cdots & N-1 & N \\ 2 & 3 & \cdots & N & N+1 \end{pmatrix}.$$

因此 $T_b(\alpha,n)$ 和 $T_b(\beta,n)$ 给出了局部 trial 和 test 基函数 $\psi_{n\alpha},\psi_{n\beta\,(\alpha,\beta=1,2)}$ 的全局节点编号.

因此对于 $n=1,2,\cdots,N$,

$$\int_{x_n}^{x_{n+1}} c\psi'_{n\alpha}\psi'_{n\beta}dx\,(\alpha,\beta=1,2).$$

需要装配到矩阵项 a_{ij} 上, 其中 $i=T_b(\beta,n), j=T_b(\alpha,n)$.

所以组装矩阵就有以下步骤:

- 初始化矩阵: $A=sparse(N_b,N_b)$;[29]
- 计算积分并将其装配进 A:

g:assemble1d

$FOR\ n=1,\cdots,N$:
  $FOR\ \alpha=1,\cdots,N_{lb}$:
    $FOR\ \beta=1,\cdots,N_{lb}$:
      Compute $r=\int_{x_n}^{x_{n+1}} c\psi'_{n\alpha}\psi'_{n\beta}\,dx$;
      Add r to $A(T_b(\beta,n),T_b(\alpha,n))$;
    END
  END
END

上述算法没有使用单刚, 直接组装总刚. 类似的有通过单刚的算法:

- Initialize the matrix: $A=sparse(N_b,N_b)$ and $S=zeros(N_{lb},N_{lb})$;
- Compute the integrals and assemble them into A:

$FOR\ n=1,\cdots,N$:
  $FOR\ \alpha=1,\cdots,N_{lb}$:
    $FOR\ \beta=1,\cdots,N_{lb}$:
      Compute $S(\beta,\alpha)=\int_{x_n}^{x_{n+1}} c\psi'_{n\alpha}\psi'_{n\beta}\,dx$;
    END
  END
  $A(T_b(:,n),T_b(:,n))=A(T_b(:,n),T_b(:,n))+S$;
END

1d_emt_stiff

问题在于上述算法只考虑了一阶导数的情况, 需要考虑

$$\int_{x_n}^{x_{n+1}} c\psi^{(r)}_{n\alpha}\psi^{(s)}_{n\beta}dx,$$

[28]有限元基函数和节点一一对应所以比较简单.

[29]sparse 是 matlab 中内置的构造稀疏矩阵的函数, 在大规模计算时为保证存储足够, 需要使用该函数不存零项.

其中参数 $r, s > 1$ 的情形该如何处理. 可以考虑将导数阶设计成参数实现自动化处理.[30]
其次对于 Petrov 有限元, trial 函数空间和 test 函数空间是不同的. 即不妨设 trial 函数空间为 $span\{\varphi_{n\alpha}\}$, test 函数空间为 $span\{\psi_{n\beta}\}$, 则有一个更一般的积分形式

$$\int_{x_n}^{x_{n+1}} c\varphi_{n\alpha}^{(r)}\psi_{n\beta}^{(s)}dx.$$

同时对于该问题的系数 $c(x)$ 希望也能将其设计成参数, 实现自动化处理.
因此需要将以下部分设计参数化:

- 系数函数 c;
- 关于数值积分的高斯积分节点和权重;
- 网格信息矩阵 P, T, 同时也可以知道网格单元的个数 $N = size(T, 2)$ 以及网格节点 $N_m = size(P, 2)$.
- 关于 trial 函数和 test 函数的基函数类型.
- 有限元信息矩阵 P_b, T_b, 这同时给出了局部基函数的个数 $N_{lb} = size(T_b, 1)$ 以及全局基函数个数 $N_b = size(P_b, 2)$.[31]

Algorithm 4 (upgraded version):

- Initialize the matrix: $A = sparse(N_b^{test}, N_b^{trial})$;
- Compute the integrals and assemble them into A:

```
FOR n = 1, ··· , N:
    FOR α = 1, ··· , N_lb^trial:
        FOR β = 1, ··· , N_lb^test:
            Compute r = ∫_{x_n}^{x_{n+1}} c φ_{nα}^{(r)} ψ_{nβ}^{(s)} dx;
            Add r to A(T_b(β, n), T_b(α, n));
        END
    END
END
```

g:alg:1dfinal

载荷向量装配类似于刚度矩阵, 此处略. 详细内容可以参考讲义 chapter1.page65-75.
具体算法:

Algorithm 5 (upgraded version):

- Initialize the vector: $b = zeros(N_b, 1)$;
- Compute the integrals and assemble them into $\vec{b}$:

```
FOR n = 1, ··· , N:
    FOR β = 1, ··· , N_lb:
        Compute r = ∫_{x_n}^{x_{n+1}} f ψ_{nβ}^{(s)} dx;
        b(T_b(β, n), 1) = b(T_b(β, n), 1) + r;
    END
END
```

g:alg:1dload

[30] 虽然这样会导致程序变复杂运行速度下降.
[31] 当使用 Petrov 有限元时会有对应 trial 函数和 test 函数的不同有限元信息矩阵.

2.5.6 施加边界条件

- 边界信息矩阵 $boundarynodes$;
- $boundarynodes(1,k)$ 存储的是第 k 个边界点的类型: Dirichlet,Neumann,Robin ...
- $boundarynodes(2,k)$ 存储的是第 k 个边界点的全局节点编号.
- 设 nbn 是有限元边界节点的个数.
- 定义 $g(x)$ 是满足 $g(a)=g_a$和$g(b)=g_b$ 的边界函数.

施加边界条件 (仅考虑 Dirichlet 边界条件):

Algorithm 6:

- Deal with the Dirichlet boundary conditions:

$FOR\ k=1,\cdots,nbn$:
  $IF\ boundarynodes(1,k)$ shows Dirichlet condition,
$THEN$
    $i=boundarynodes(2,k)$;
    $A(i,:)=0$;
    $A(i,i)=1$;
    $b(i)=g(P_b(i))$;
  $ENDIF$
END

g:1ddirichlet

程序设计总流程

- Input a, b, and N. Generate the mesh information matrices P and T, the finite element information matrices P_b and T_b for the trial and test functions respectively.
- Assemble the stiffness matrix A by using Algorithm 4.
- Assemble the load vector $\vec{b}$ by using Algorithm 5.
- Deal with the Drichlet boundary condition by using Algorithm 6.
- Solve $A\vec{X}=\vec{b}$ for $\vec{X}$ by using a direct or iterative method.

lg:alg:1dcode

2.6 推广

2.7 总结

A 2020.12.13 第一次编程课

可以直接跟随视频编程, 主要是 1d 整体框架的搭建. 有限元编程何晓明—第一次编程课